

OSGi Working Group
OSGi Specification repo for
`org.osgi.service.dalRelease`

1.0.0-SNAPSHOT

September 2026



LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>"

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
141	Device Abstraction Layer Specification	7
141.1	Introduction	7
141.2	Device Category	8
141.3	Device Service	8
141.4	Function Service	18
141.5	Security	22
141.6	org.osgi.service.dal	23
141.7	References	48

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

DRAFT

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

141 Device Abstraction Layer Specification

Version 1.0

141.1 Introduction

The Internet-of-Things (IoT) has a major impact in the IT industry. It requires backend systems to receive information from sensors, actuators, and appliances in various vertical markets such as Smart Home, eHealth, industrial automation, logistics, and automotive telematics. Application developers have to face the still increasing amount of communication protocols which are the major hurdle for interoperability.

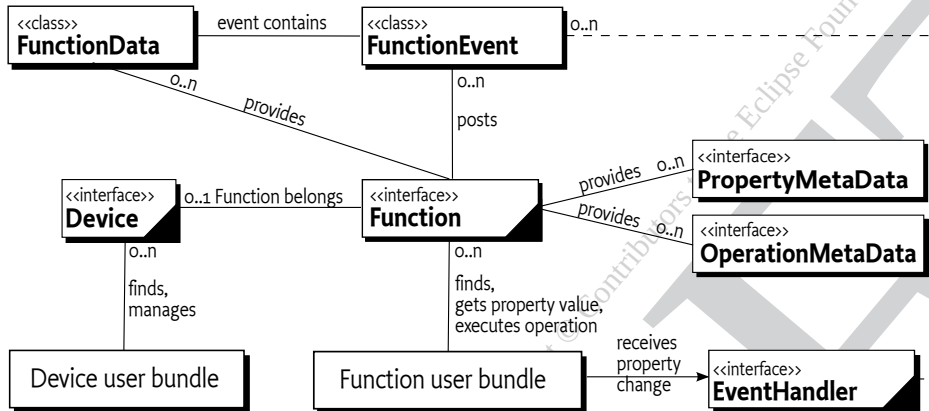
The Device Abstraction Layer specification provides a unified interface for application developers to interact with sensor, devices, etc. connected to a gateway. Application developers don't have to deal with protocol specific details which simplifies the development of their applications.

The remote device control provides an opportunity to save energy, to support better security, to save your time during daily tasks and more. The devices can play different roles in their networks as event reporters, controllers, etc. That dynamic behavior is well mappable to the dynamic OSGi service registry. When a new device is available in the network, there is a registration of a Device service. It realizes basic set of management operations and provides a rich set of properties. The applications can track the device status, read descriptive information and follow the device relations. A set of functions can belong to a single device. They represent the device operations and related properties in an atomic way. The device functions can be found in the OSGi service registry. The applications are allowed to get directly the required functions if they don't need information about the device. For example, light device is registered as Device service and there is Function service to turn on and turn off the light. The application can operate with the light control service without access to the device service.

141.1.1 Entities

- *Device* - represents the device in the OSGi service registry. It's described with a set of service properties and provides basic management operations.
- *Function* - atomic functional entity like switch or sensor. The function can belong to a device. The function provides a set of properties and operations.
- *FunctionEvent* - asynchronous event. It's posted through EventAdmin service and notifies for Function property change.
- *FunctionData* - data structure which carries Function property value with extra metadata.
- *PropertyMetadata* and *OperationMetadata* - contain metadata about the Function properties and operations.

Figure 141.1 Device Abstraction Layer Overview



141.2 Device Category

The device category defined in the scope of the Device Access service specification is called DAL. `DEVICE_CATEGORY` constant contains the category name.

141.3 Device Service

The `Device` interface is dedicated to a common access to the devices provided by different protocols. It can be mapped one to one with the physical device, but can be mapped only with a given functional part of the device. Another mapping can be a device realized with a set of `Device` services and different relations between them. `Device` service can represent pure software unit. For example, it can simulate the real device work. There are basic management operations for removal and property access. New protocol devices can be supported with the registration of new `Device` services.

If the underlying protocol and the implementation allow, the `Device` services must be registered again after the OSGi framework restarts. The service properties must be restored, the supported functions must be registered and `Device` relations must be visible to the applications.

141.3.1 Device Service Properties

The OSGi service registry has the advantage of being easily accessible. The services can be filtered and accessed with their properties. The `Device` service has a rich set of such properties:

- `SERVICE_UID` – Specifies the device unique identifier. It's a mandatory property. The value type is `java.lang.String`. To simplify the unique identifier generation, the property value must follow the rule:

```
UID ::= driver-name ':' device-id
```

- `UID` – device unique identifier
- `driver-name` – the value of the `Device.SERVICE_DRIVER` service property
- `device-id` – device unique identifier in the scope of the driver
- `SERVICE_REFERENCE_UIDS` – Specifies the reference device unique identifiers. It's an optional property. The value type is `java.lang.String[]`. It can be used to represent different relationships

between the devices. For example, The EnOcean controller can have a reference to the USB dongle.

- **SERVICE_DRIVER** – Specifies the device driver name. For example, EnOcean, Z-Wave, Bluetooth, etc. It's a mandatory property. The value type is `java.lang.String`.
- **SERVICE_NAME** – Specifies the device name. It's an optional property. The value type is `java.lang.String`.
- **SERVICE_STATUS** – Specifies the current device status. It's a mandatory property. The value type is `java.lang.Integer`. The possible values are:
 - **STATUS_REMOVED** – Indicates that the device has been removed from the network. That status must be set as the last device status and after that the device service can be unregistered from the service registry. The status is available for stale device services too. All transitions to this status are described in *Removed* on page 13 .
 - **STATUS_OFFLINE** – Indicates that the device is currently not available for operations. The end device is available in the network and can become online later. The controller is unplugged or there is no connection. All transitions to and from this status are described in detail in *Offline* on page 13 .
 - **STATUS_ONLINE** – Indicates that the device is currently available for operations. The recent communication with the device has been passed through. All transitions to and from this status are described in detail in *Online* on page 14 .
 - **STATUS_PROCESSING** – Indicates that the device is currently busy with an operation. All transitions to and from this status are described in detail in *Processing* on page 15 .
 - **STATUS_NOT_INITIALIZED** – Indicates that the device is currently not initialized. Some protocols don't provide device information right after the device is connected. The device can be initialized later when it's awakened. All transitions to and from this status are described in detail in *Not Initialized* on page 16 .
 - **STATUS_NOT_CONFIGURED** – Indicates that the device is currently not configured. The device can require additional actions to become completely connected to the network. All transitions to and from this status are described in detail in *Not Configured* on page 17 .
- **SERVICE_STATUS_DETAIL** – Provides the reason for the current device status. It's an optional property. The property value cannot be externally set or modified. The value type is `java.lang.Integer`. There are two value categories. Positive values indicate the reason for the current status like **STATUS_DETAIL_CONNECTING**. Negative values indicate errors related to the current device status like **STATUS_DETAIL_BROKEN**. The list with defined status details is:
 - **STATUS_DETAIL_CONNECTING** – The device is currently connecting to the network. The status detail indicates the reason with a positive value 1. The device status must be **STATUS_PROCESSING**.
 - **STATUS_DETAIL_INITIALIZING** – The device is currently in process of initialization. The status detail indicates the reason with a positive value 2. The network controller initializing means that information about the network is currently read. The device status must be **STATUS_PROCESSING**.
 - **STATUS_DETAIL_REMOVING** – The device is leaving the network. The status detail indicates the reason with positive value 3. The device status must be **STATUS_PROCESSING**.
 - **STATUS_DETAIL_FIRMWARE_UPDATING** – The device firmware is updating. The status detail indicates the reason with positive value 4. The device status must be **STATUS_PROCESSING**.
 - **STATUS_DETAIL_CONFIGURATION_UNAPPLIED** – The device configuration is not applied. The status detail indicates an error with a negative value -1. The device status must be **STATUS_NOT_CONFIGURED**.
 - **STATUS_DETAIL_BROKEN** – The device is broken. The status detail indicates an error with a negative value -2. The device status must be **STATUS_OFFLINE**.
 - **STATUS_DETAIL_COMMUNICATION_ERROR** – The device communication is problematic. The status detail indicates an error with a negative value -3. The device status must be **STATUS_ONLINE** or **STATUS_NOT_INITIALIZED**.

- `STATUS_DETAIL_DATA_INSUFFICIENT` – The device doesn't provide enough information and cannot be determined. The status detail indicates an error with a negative value -4. The device status must be `STATUS_NOT_INITIALIZED`.
- `STATUS_DETAIL_INACCESSIBLE` – The device is not accessible and further communication is not possible. The status detail indicates an error with a negative value -5. The device status must be `STATUS_OFFLINE`.
- `STATUS_DETAIL_CONFIGURATION_ERROR` – The device cannot be configured. The status detail indicates an error with a negative value -6. The device status must be `STATUS_NOT_CONFIGURED`.
- `STATUS_DETAIL_DUTY_CYCLE` – The device is in duty cycle. The status detail indicates an error with a negative value -7. The device status must be `STATUS_OFFLINE`.

Custom status details are allowed, but they must not overlap the specified codes. To prevent possible collisions with further updates, custom codes must be greater than 100 and less than -100. Table 141.1 contains the mapping of the status details to the statuses.

Table 141.1 *Status detail to status mapping.*

Status Detail	Status
CONNECTING	PROCESSING
INITIALIZING	PROCESSING
REMOVING	PROCESSING
FIRMWARE_UPDATING	PROCESSING
CONFIGURATION_UNAPPLIED	NOT_CONFIGURED
BROKEN	OFFLINE
COMMUNICATION_ERROR	ONLINE, NOT_INITIALIZED
DATA_INSUFFICIENT	NOT_INITIALIZED
INACCESSIBLE	OFFLINE
CONFIGURATION_ERROR	NOT_CONFIGURED
DUTY_CYCLE	OFFLINE
• <code>SERVICE_HARDWARE_VENDOR</code> – Specifies the device hardware vendor. It's an optional property. The value type is <code>java.lang.String</code> .	
• <code>SERVICE_HARDWARE_VERSION</code> – Specifies the device hardware version. It's an optional property. The value type is <code>java.lang.String</code> .	
• <code>SERVICE_FIRMWARE_VENDOR</code> – Specifies the device firmware vendor. It's an optional property. The value type is <code>java.lang.String</code> .	
• <code>SERVICE_FIRMWARE_VERSION</code> – Specifies the device firmware version. It's an optional property. The value type is <code>java.lang.String</code> .	
• <code>SERVICE_TYPES</code> – Specifies the device types. It's an optional property. The value type is <code>java.lang.String[]</code> .	
• <code>SERVICE_MODEL</code> – Specifies the device model. It's an optional property. The value type is <code>java.lang.String</code> .	
• <code>SERVICE_SERIAL_NUMBER</code> – Specifies the device serial number. It's an optional property. The value type is <code>java.lang.String</code> .	

The next code snippet prints all online devices.

```
ServiceReference[] deviceSRefs = context.getServiceReferences(  
    Device.class.getName(),  
    '(' + Device.SERVICE_STATUS + '=' + Device.STATUS_ONLINE + ')');  
if (deviceSRefs != null) {  
    for (int i = 0; i < deviceSRefs.length; i++) {
```

```

        printDevice(deviceSRefs[i]);
    }
}

```

Applications need to have an access to the device properties. For convenience, there are helper methods:

- [getServiceProperty\(String\)](#) – Returns the current value of the specified property. The method will return the same value as `org.osgi.framework.ServiceReference.getProperty(String)` for the service reference of this device.
- [getServicePropertyKeys\(\)](#) – Returns an array with all device service property keys. The method will return the same value as `org.osgi.framework.ServiceReference.getPropertyKeys()` for the service reference of this device.

141.3.2

Device Registration

The devices are registered as services in the OSGi service registry. The service interface is `org.osgi.service.dal.Device`. There is a registration order. [Device](#) services are registered last on start up. Before their registration, there is [Function](#) service registration. The function registration procedure is described in *Function Registration* on page 19 .

The OSGi service registry provides an access to the services, but there are no management operations like remove a given service. The service provider is responsible to register and unregister own services. That design doesn't provide an option to remove the device services. The [Device](#) interface fills this gap with [remove\(\)](#) method. It's a callback to the service provider to remove the device from the network. The method can be optionally implemented. `java.lang.UnsupportedOperationException` can be thrown if the method is not supported. When the [remove\(\)](#) is called:

- An appropriate command will be synchronously send to the device. As a result it can leave the network.
- The device status will be set to [STATUS_REMOVED](#) .
- The related device service will be unregistered from the OSGi service registry.

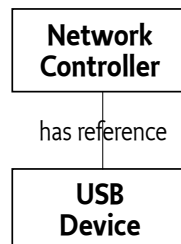
There is an unregistration order. The registration reverse order is used when the services are unregistered. [Device](#) services are unregistered first before [Function](#) services.

141.3.3

Reference Devices

[Device](#) service can have a reference to other devices. That link can be used to represent different relationships between devices. For example, the EnOcean dongle can be used as USB Device and EnOcean network controller [Device](#). The network controller device can have a reference to the physical USB device as it's depicted on the next diagram.

Figure 141.2 Device Reference



The related service property is [SERVICE_REFERENCE_UIDS](#) .

141.3.4

Device Status Transitions

The device status reveals the device availability. It can demonstrate that device is currently not available for operations or that the device requires some additional configuration steps. The status can move between the different values according to the rules defined in this section. The status transitions are summarized in Table 141.2 , visualized on Figure 141.3 and described in detail in the next sections. The initial device status is always `STATUS_PROCESSING` . When device info is processed, the device can go to another status. The last possible device status is `STATUS_REMOVED` . The status must be set when the device is removed from the network. After that status, the device service will be unregistered.

Figure 141.3

Device Status Transitions

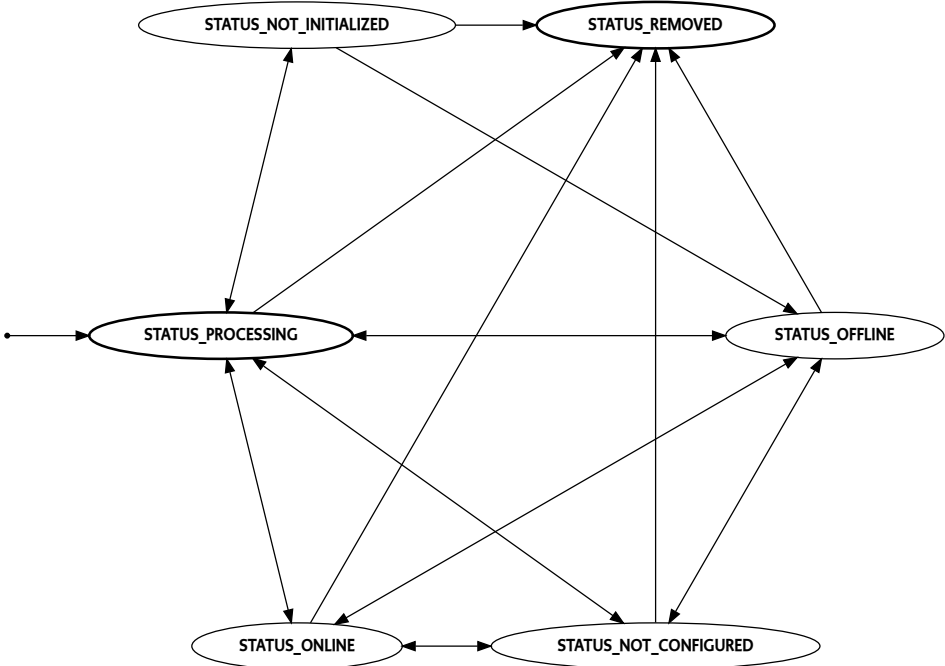


Table 141.2

Device Status Transitions

From\To Sta- tus	PRO- CESSING	ONLINE	OFFLINE	NOT INITIALIZED	NOT CON- FIGURED	REMOVED
PRO- CESSING	-	Initial de- vice data has been read.	Device is not accessible.	Initial de- vice data has been partial- ly read.	Device has a pending con- figuration.	Device has been re- moved.
ONLINE	Device da- ta is process- ing.	-	Device is not accessible.	-	Device has a new pend- ing configu- ration.	Device has been re- moved.
OFFLINE	Device da- ta is process- ing.	Device da- ta has been read.	-	-	Device has a pending con- figuration.	Device has been re- moved.
NOT INITIALIZED	Device da- ta is process- ing.	-	Device is not accessible.	-	-	Device has been re- moved.

From\To Status	PRO-CESSING	ONLINE	OFFLINE	NOT INITIALIZED	NOT CON-FIGURED	REMOVED
NOT CON-FIGURED	Device data is processing.	Device pending configuration is satisfied.	Device is not accessible.	-	-	Device has been removed.
REMOVED	-	-	-	-	-	-

141.3.4.1**Removed**

The device can go to [STATUS_REMOVED](#) from any other status. Once reached, the device status cannot be updated any more. The device has been removed from the network and the device service is unregistered from the OSGi service registry. If there are stale references to the [Device](#) service, their status will be set to [STATUS_REMOVED](#).

The common way for a given device to be removed is [remove\(\)](#) method. When the method returns, the device status will be [STATUS_REMOVED](#). It requires a synchronous execution of the operation.

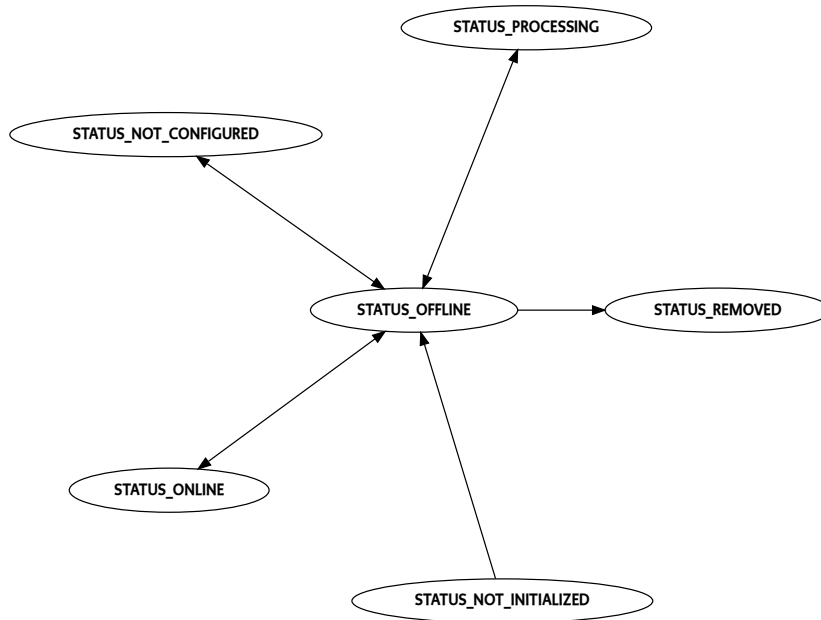
141.3.4.2**Offline**

The [STATUS_OFFLINE](#) indicates that the device is currently not available for operations. That status can be set, because of different reasons. The network controller has been unplugged, the connection to the device has been lost, etc. The device can move to this status from any other status with the exception of [STATUS_REMOVED](#). Transitions to and from this status are:

- From [STATUS_OFFLINE](#) to [STATUS_REMOVED](#) – The device has been removed. The status can be set as a result of [remove\(\)](#) method call.
- From [STATUS_OFFLINE](#) to [STATUS_PROCESSING](#) – Device data is processing.
- From [STATUS_OFFLINE](#) to [STATUS_NOT_CONFIGURED](#) – The device has a pending configuration.
- From [STATUS_OFFLINE](#) to [STATUS_ONLINE](#) – Device data has been read and the device is currently available for operations.
- From [STATUS_OFFLINE](#) to [STATUS_NOT_INITIALIZED](#) – That transition is not possible, because the status have to go through [STATUS_PROCESSING](#). If the processing is unsuccessful, [STATUS_NOT_INITIALIZED](#) will be set.
- To [STATUS_OFFLINE](#) from [STATUS_REMOVED](#) – That transition is not possible. If the device has been removed, the service will be unregistered from the service registry.
- To [STATUS_OFFLINE](#) from [STATUS_PROCESSING](#) – The device is not accessible any more while device data is processing.
- To [STATUS_OFFLINE](#) from [STATUS_NOT_CONFIGURED](#) – The device with pending configuration is not accessible any more.
- To [STATUS_OFFLINE](#) from [STATUS_ONLINE](#) – The online device is not accessible any more.
- To [STATUS_OFFLINE](#) from [STATUS_NOT_INITIALIZED](#) – The not initialized device is not accessible any more.

The possible transitions are summarized on Figure 141.4.

Figure 141.4

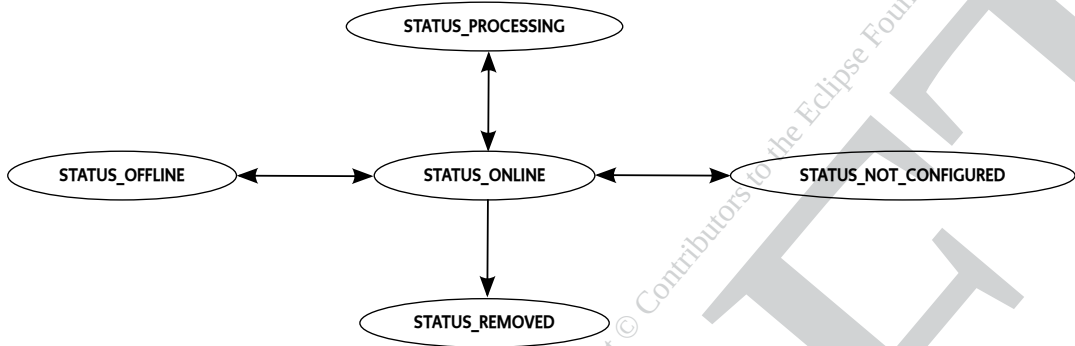
Transitions to and from `STATUS_OFFLINE`**141.3.4.3****Online**

The `STATUS_ONLINE` indicates that the device is currently available for operations. The online devices are initialized and ready for use. Transitions to and from this status are:

- From `STATUS_ONLINE` to `STATUS_REMOVED` – The device has been removed. The status can be set as a result of `remove()` method call.
- From `STATUS_ONLINE` to `STATUS_PROCESSING` – The device data is processing.
- From `STATUS_ONLINE` to `STATUS_NOT_CONFIGURED` – The device has a pending configuration.
- From `STATUS_ONLINE` to `STATUS_OFFLINE` – The online device is not accessible any more.
- From `STATUS_ONLINE` to `STATUS_NOT_INITIALIZED` – That transition is not possible. Online devices are initialized.
- To `STATUS_ONLINE` from `STATUS_REMOVED` – That transition is not possible. If the device has been removed, the service will be unregistered from the service registry.
- To `STATUS_ONLINE` from `STATUS_PROCESSING` – Initial device data has been read. The device is available for operations.
- To `STATUS_ONLINE` from `STATUS_NOT_CONFIGURED` – The device pending configuration is satisfied.
- To `STATUS_ONLINE` from `STATUS_OFFLINE` – The device is accessible for operations.
- To `STATUS_ONLINE` from `STATUS_NOT_INITIALIZED` – That transition is not possible. The device data has to be processed and then the device can become online. Intermediate status `STATUS_PROCESSING` will be used.

The possible transitions are summarized on Figure 141.5.

Figure 141.5 Transitions to and from STATUS_ONLINE

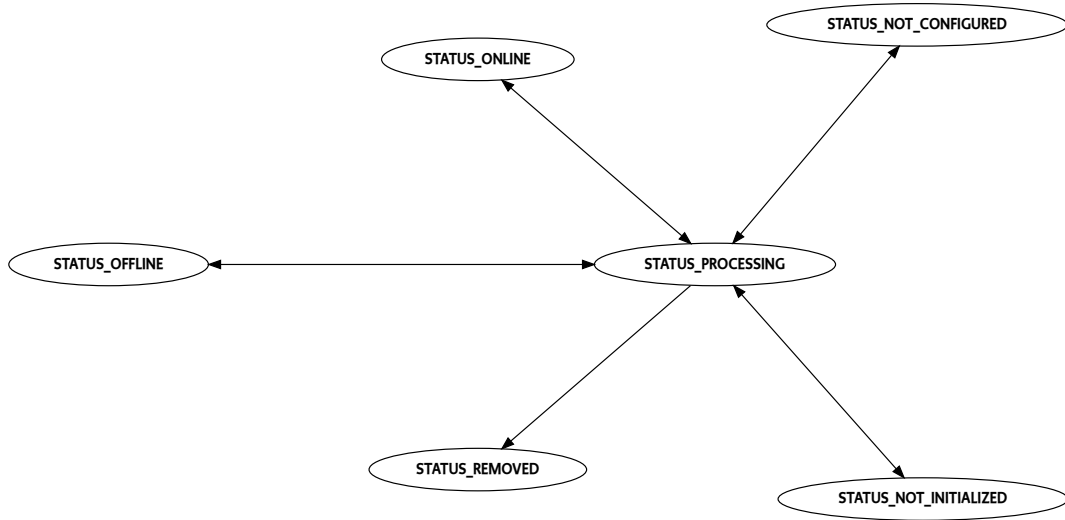
**141.3.4.4****Processing**

The status indicates that the device is currently busy with an operation. It can be time consuming operation and can result to any other status. The operation processing can be reached by any other status except **STATUS_REMOVED**. For example, offline device requires some data processing to become online. It will apply this status sequence: **STATUS_OFFLINE**, **STATUS_PROCESSING** and **STATUS_ONLINE**. Transitions to and from this status are:

- From **STATUS_PROCESSING** to **STATUS_REMOVED** – The device has been removed. The status can be set as a result of `remove()` method call.
- From **STATUS_PROCESSING** to **STATUS_ONLINE** – Initial device data has been read. The device is available for operations.
- From **STATUS_PROCESSING** to **STATUS_NOT_CONFIGURED** – The device has a pending configuration.
- From **STATUS_PROCESSING** to **STATUS_OFFLINE** – The device is not accessible any more.
- From **STATUS_PROCESSING** to **STATUS_NOT_INITIALIZED** – The device initial data is partially read.
- To **STATUS_PROCESSING** from **STATUS_REMOVED** – That transition is not possible. If the device has been removed, the service will be unregistered from the service registry.
- To **STATUS_PROCESSING** from **STATUS_ONLINE** – The device is busy with an operation.
- To **STATUS_PROCESSING** from **STATUS_NOT_CONFIGURED** – The device pending configuration is satisfied and the device is busy with an operation.
- To **STATUS_PROCESSING** from **STATUS_OFFLINE** – The device is busy with an operation.
- To **STATUS_PROCESSING** from **STATUS_NOT_INITIALIZED** – The device initial data is processing.

The possible transitions are summarized on Figure 141.6.

Figure 141.6

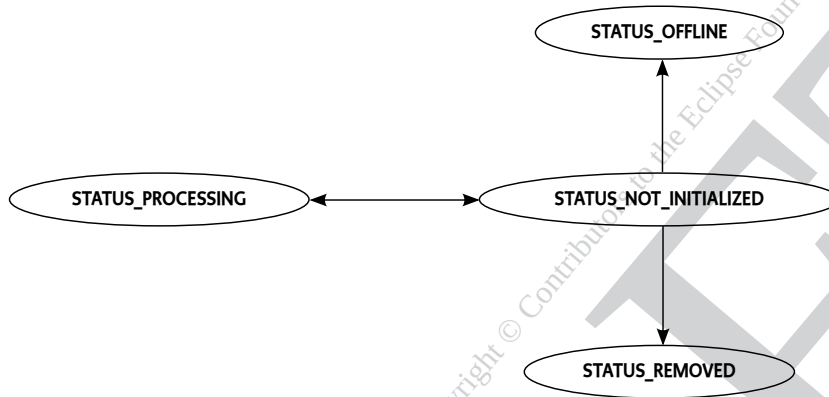
Transitions to and from `STATUS_PROCESSING`**141.3.4.5****Not Initialized**

The status indicates that the device is currently not initialized. Some protocols don't provide device information right after the device is connected. The device can be initialized later when it's awakened. The not initialized device requires some data processing to become online. `STATUS_PROCESSING` is used as an intermediate status. Transitions to and from this status are:

- From `STATUS_NOT_INITIALIZED` to `STATUS_REMOVED` – The device has been removed. The status can be set as a result of `remove()` method call.
- From `STATUS_NOT_INITIALIZED` to `STATUS_PROCESSING` – The device data is processing.
- From `STATUS_NOT_INITIALIZED` to `STATUS_NOT_CONFIGURED` – That transition is not possible. Device requires some data processing.
- From `STATUS_NOT_INITIALIZED` to `STATUS_OFFLINE` – The device is not accessible any more.
- From `STATUS_NOT_INITIALIZED` to `STATUS_ONLINE` – That transition is not possible. Device requires some data processing to become online.
- To `STATUS_NOT_INITIALIZED` from `STATUS_REMOVED` – That transition is not possible. If the device has been removed, the service will be unregistered from the service registry.
- To `STATUS_NOT_INITIALIZED` from `STATUS_PROCESSING` – Device data is partially read.
- To `STATUS_NOT_INITIALIZED` from `STATUS_NOT_CONFIGURED` – That transition is not possible. When device pending configuration is satisfied, the device requires additional data processing.
- To `STATUS_NOT_INITIALIZED` from `STATUS_OFFLINE` – That transition is not possible. Device requires some data processing and then can become not initialized.
- To `STATUS_NOT_INITIALIZED` from `STATUS_ONLINE` – That transition is not possible. The online device is initialized.

The possible transitions are summarized on Figure 141.7.

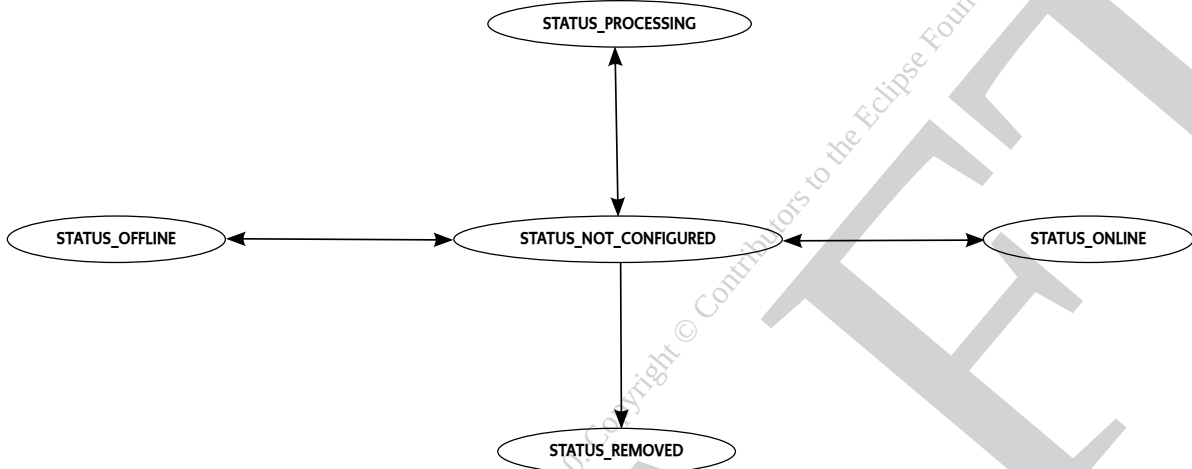
Figure 141.7 Transitions to and from STATUS_NOT_INITIALIZED

**141.3.4.6****Not Configured**

Indicates that the device is currently not configured. The device can require additional actions to become completely connected to the network. For example, a given device button has to be pushed. That status doesn't have transitions with `STATUS_NOT_INITIALIZED`, because some data processing is required. Transitions to and from this status are:

- From `STATUS_NOT_CONFIGURED` to `STATUS_REMOVED` – The device has been removed. The status can be set as a result of `remove()` method call.
- From `STATUS_NOT_CONFIGURED` to `STATUS_PROCESSING` – The device pending configuration is satisfied and some additional data processing is required.
- From `STATUS_NOT_CONFIGURED` to `STATUS_ONLINE` – The device pending configuration is satisfied.
- From `STATUS_NOT_CONFIGURED` to `STATUS_OFFLINE` – The device is not accessible any more.
- From `STATUS_NOT_CONFIGURED` to `STATUS_NOT_INITIALIZED` – That transition is not possible. When device pending configuration is satisfied, the device requires additional data processing.
- To `STATUS_NOT_CONFIGURED` from `STATUS_REMOVED` – That transition is not possible. If the device has been removed, the service will be unregistered from the service registry.
- To `STATUS_NOT_CONFIGURED` from `STATUS_PROCESSING` – Initial device data has been read but there is a pending configuration.
- To `STATUS_NOT_CONFIGURED` from `STATUS_ONLINE` – The device has a pending configuration.
- To `STATUS_NOT_CONFIGURED` from `STATUS_OFFLINE` – The device is going to be online, but has a pending configuration.
- To `STATUS_NOT_CONFIGURED` from `STATUS_NOT_INITIALIZED` – That transition is not possible. Device requires some data processing.

The possible transitions are summarized on Figure 141.8.

Figure 141.8 Transitions to and from `STATUS_NOT_CONFIGURED`

141.4 Function Service

The user applications have full control over the device with the [Function](#) services. Synchronous or asynchronous operations can trigger different actions. For example, turn on or off the light, can change the room temperature, send an user notification, etc. The action result can be reported immediately or later in case of concurrent execution. As a result, a [Function](#) property can be updated. The property is the device value container. It can provide, sensor information, meter data, the switch current position, etc. Different property access types allow the applications to read, write or receive events.

141.4.1 Function Service Properties

The OSGi service registry has the advantage of being easily accessible. The services can be filtered and accessed with their properties. The function service has a rich set of such properties:

- [SERVICE_UID](#) – mandatory service property. The property value is the function unique identifier. The value type is `java.lang.String`. To simplify the unique identifier generation, the property value must follow the rule:

```
function UID ::= device-id ':' function-id
```

- function UID – function unique identifier
- device-id – the value of the `Device.SERVICE_UID` Device service property
- function-id – function identifier in the scope of the device

If the function is not bound to a device, the function unique identifier can be device independent.

- [SERVICE_TYPE](#) – optional service property. The service property value contains the function type. For example, the sensor function can have different types like temperature, pressure, etc. The value type is `java.lang.String`.

Organizations that want to use function types that do not clash with OSGi Working Group defined types should prefix their types in own namespace.

- [SERVICE_VERSION](#) – optional service property. The service property value contains the function version. That version can point to specific implementation version and vary in the different vendor implementations. The value type is `java.lang.String`.

- **SERVICE_DEVICE_UID** – optional service property. The property value is the device identifier. The function belongs to this device. The value type is `java.lang.String`.
- **SERVICE_REFERENCE_UIDS** – optional service property. The service property value contains the reference function unique identifiers. The value type is `java.lang.String[]`. It can be used to represent different relationships between the functions.
- **SERVICE_DESCRIPTION** – optional service property. The property value is the function description. The value type is `java.lang.String`.
- **SERVICE_OPERATION_NAMES** – optional service property. The property is missing when there are no function operations and property must be set when there are function operations. The property value is the function operation names. The value type is `java.lang.String[]`. It's not possible to exist two or more function operations with the same name i.e. the operation overloading is not allowed.
- **SERVICE_PROPERTY_NAMES** – optional service property. The property is missing when there are no function properties and property must be set when there are function properties. The property value is the function property names. The value type is `java.lang.String[]`. It's not possible to exist two or more function properties with the same name.

141.4.2 Function Registration

On start up, the Function services are registered before the **Device** service. It's possible that **SERVICE_DEVICE_UID** points to missing service at the moment of the registration. The reverse order is used when the services are unregistered. **Device** service is unregistered before the **Function** services. The device registration procedure is available in *Device Registration* on page 11 .

The **Function** service should be registered only under the function class hierarchy. Other classes can be used if there are no ambiguous representations. For example, an ambiguous representation can be a function registered under two independent function classes like `BinarySwitch` and `Meter`. In this example, both functions support the same property “state” with different meaning. `getPropertyMetadata(String propertyName)` method cannot determinate which property is requested. It can be `BinarySwitch` “state” or `Meter` “state”.

To simplify the generic function discovery, the Function interface must be used for the service registration. In this way, the generic applications can easily find all services, which are functions in the service registry. Because of this rule, this registration is not allowed:

```
context.registerService(MeterV1.class.getName(), this, regProps);
```

If the implementation would like to mark that there is a function, but no specific function interface exists, the registration can be:

```
context.registerService(Function.class.getName(), this, regProps);
```

Note that such functions usually don't have operations and properties.

141.4.3 Function Interface

Function is built by a set of properties and operations. The function can have unique identifier, type, version, description, link to the **Device** service and information about the referenced functions. **Function** interface must be the base interface for all functions. If the device provider defines custom functions, all of them must extend **Function** interface. It provides a common access to the operations and properties metadata.

There are some general type rules, which unify the access to the function data. They make easier the transfer over different protocols. All properties and operation arguments must use one of:

- Java primitive type or corresponding reference type.
- Numerical type i.e. the type which extends `java.lang.Number`. The numerical type must follow these conventions:

- The type must provide a public static method called `valueOf` that returns an instance of the given type and takes a single `String` argument or a public constructor which takes a single `String` argument.
- The `String` argument from the previous bullet can be provided by `toString()` method of the instance.
- `java.lang.String`
- Java Bean, but its properties must use those rules. Java Bean is defined in [1] *JavaBeans Spec*.
- `java.util.Map` instance. The map keys can be `java.lang.String`. The values of a single type follow these rules.
- Array of defined types.

In order to provide common behavior, all functions must follow a set of common rules related to the implementation of their setters, getters, operations and events:

- The setter method must be executed synchronously. If the underlying protocol can return response to the setter call, it must be awaited. It simplifies the property value modification and doesn't require asynchronous callback.
- The operation method must be executed synchronously. If the underlying protocol can return an operation confirmation or response, it must be awaited. It simplifies the operation execution and doesn't require asynchronous callback.
- The getter must return the last known cached property value. The device implementation is responsible to keep that value up to date. It'll speed up the applications when the function property values are collected. The same cached value can be shared between a few requests instead of a few calls to the real device.
- The function operations, getters and setters must not override `java.lang.Object` and this interface methods. For example:
 - `hashCode()` – it's `java.lang.Object` method and invalid function operation;
 - `wait()` – it's `java.lang.Object` method and invalid function operation;
 - `getClass()` – it's `java.lang.Object` method and invalid function getter;
 - `getPropertyMetadata(String propertyName)` – it's `org.osgi.service.dal.Function` method and invalid function getter.

141.4.4 Function Operations

Function operations are the main callable units. They can perform a specific task on the device like turn on or turn off. They can be used by the applications to control the device. Operation names are available as a value of the service property `SERVICE_OPERATION_NAMES`. The operations are identified by their names. It's not possible to exist two operations with the same name i.e. overloaded operations are not allowed. They cannot override the property accessor methods. The operations are regular java methods. That implies that they have zero or more arguments and zero or one return value. The operation arguments and return value must follow the general type rules.

The operations can be optionally described with metadata. Metadata is accessible with `getOperationMetadata(String)` method. The result provides metadata about the operation, operation arguments and result value. Operation arguments and result value are using the same metadata as the function properties. The full details are defined in the next section.

141.4.5 Function Properties

Function properties are class fields. Their values can be read with getter methods and can be set with setter methods. The property names are available as a value of the service property `SERVICE_PROPERTY_NAMES`. The properties are identified by their names. It's not possible to exist two properties with the same name.

The function properties must be integrated according to these rules:

- Getter methods must be available for all properties with [ACCESS_READABLE](#) access.
 - Getter method must return a subclass of [FunctionData](#).
 - Setter methods must be available for all properties with [ACCESS_WRITABLE](#) access.
 - Setter methods can be any combination of:
 - Setter method which accepts a subclass of [FunctionData](#).
 - Setter method which accepts the values used by the [FunctionData](#) subclass, if there are no equal types.
- It's possible to have only one or both of them. Examples:
- There is `MyFunctionData` bean with `BigDecimal` value for a data property. Valid setters are `setData(MyFunctionData data)` and `setData(BigDecimal data)`.
 - There is `MySecondFunctionData` bean with `BigDecimal` prefix and `BigDecimal` suffix for a data property. The prefix and suffix are using equal types and we cannot have a setter with the values used by `MySecondFunctionData`. The only one possible setter is `setData(MySecondFunctionData data)`.
 - No methods are required for properties with [ACCESS_EVENTABLE](#) access.

The accessor method names must be defined according to [1] *JavaBeans Spec*.

The properties can be optionally described with a set of metadata properties. The property values can be collected with [getPropertyMetadata\(String\)](#) method. The method result is [PropertyMetadata](#) with:

- Minimum value – available through [getMinValue\(String\)](#). The minimum value can be different for the different units.
- Maximum value – available through [getMaxValue\(String\)](#). The maximum value can be different for the different units.
- Enumeration of values – available through [getEnumValues\(String\)](#). The array of the possible values is sorted in increasing order according to the given unit.
- Step – available through [getStep\(String\)](#). The difference between two values in series. For example, if the range is `[0, 100]`, the step can be 10.
- Property access – available as a value in [getMetadata\(String\)](#) result map. It's a bitmap of `java.lang.Integer` type and doesn't depend on the given unit. The access is available only for the function properties and it's missing for the operation arguments and result metadata. The bitmap can be any combination of:
 - [ACCESS_READABLE](#) – Marks the property as a readable. Function must provide a getter method for this property according to [1] *JavaBeans Spec*. Function operations must not be overridden by this getter method.
 - [ACCESS_WRITABLE](#) – Marks the property as writable. Function must provide a setter method for this property according to [1] *JavaBeans Spec*. Function operations must not be overridden by this setter method.
 - [ACCESS_EVENTABLE](#) – Marks the property as eventable. Function must not provide special methods because of this access type. `FunctionEvent` is sent on property change. Note that the event can be sent when there is no value change.
- Units - available as a value in [getMetadata\(String\)](#) result map. They can be requested with key [UNITS](#). The value contains the property supported units. The property value type is `java.lang.String[]`. The array first element at index 0 represents the default unit. Each unit must follow those rules:
 - The International System of Units must be used where it's applicable. For example, kg for kilogram and km for kilometer.
 - If the unit name matches to a Unicode symbol name, the Unicode symbol must be used. For example, the degree unit matches to the Unicode degree sign (°).

- If the unit name doesn't match to a Unicode symbol, the unit symbol must be built by Unicode Basic Latin block of characters, superscript and subscript characters. For example, watt per square meter steradian is built by $W/(m^2 sr)$.

If those rules cannot be applied to the unit symbol, custom rules are allowed.

A set of predefined unit symbols are available in `SIUnits` interface.

- Description – available as a value in `getMetadata(String)` result map. It can be requested with key `DESCRIPTION`. The property value type is `java.lang.String` and specifies a user readable description. It doesn't depend on the given unit.
- Vendor custom properties – available as a value in `getMetadata(String)` result map and can depend on the given unit. Organizations that want to use custom keys that do not clash with OSGi Working Group defined should prefix their keys in own namespace.

141.4.6 Function Property Events

The eventable function properties can trigger a new event on each property value modification. It doesn't require a modification of the value. For example, the motion sensor can send a few events with no property value change when motion is detected and continued to be detected. The event must use `FunctionEvent` class. The event properties are:

- `FUNCTION_UID` – the event source function unique identifier.
- `PROPERTY_NAME` – the property name.
- `PROPERTY_VALUE` – the property value.

For example, there is function with an eventable boolean property called “state”. When “state” value is changed to false, function implementation can post:

```
FunctionEvent {
    dal.function.UID=acme.function
    dal.function.property.name="state"
    dal.function.property.value=ACMEFunctionData(java.lang.Boolean.FALSE...)
}
```

141.5 Security

141.5.1 Device Permission

The `DevicePermission` controls the bundle's authority to perform specific privileged administrative operations on the devices. There is only one action for this permission `REMOVE` to protect `remove()` method.

The name of the permission is a filter based. For more details about filter based permissions, see OSGi Core Specification, Filter Based Permissions. The filter provides an access to all device service properties. Filter attribute names are processed in a case sensitive manner. For example, the operator can give a bundle the permission to only manage devices of vendor “acme”:

```
org.osgi.service.dal.DevicePermission("dal.device.hardware.vendor=acme", "remove")
```

The permission action allows the operator to assign only the necessary permissions to the bundle. For example, the management bundle can have permission to remove all registered devices:

```
org.osgi.service.dal.DevicePermission("*", "remove")
```

The code that needs to check the device permission must always use the constructor that takes the device as a parameter `Device` with a single action. For example, the implementation of `remove()` method must check that the caller has an access to the operation:

```

public class DeviceImpl implements Device {
    ...
    public void remove() {
        securityManager.checkPermission(
            new DevicePermission(this, DevicePermission.REMOVE));
    }
    ...
}

```

141.5.2 Required Permissions

The Device implementation must check the caller for the appropriate DevicePermission before execution of the remove operation. Once the DevicePermission is checked against the caller the implementation will proceed with the actual operation. The operation can require a number of other permissions to complete. The implementation must isolate the caller from such permission checks by use of proper privileged blocks.

DevicePermission check will keep the Device implementation in the call stack. This requires the implementation to have this permission to perform the operation. The security policy should be aware of this and should grant the correct permissions. Note that the DevicePermission is a filter based permission, see OSGi Core Specification, Filter Based Permissions. It provides flexibility and fine control based on the Device service properties.

141.6 org.osgi.service.dal

Device Abstraction Layer Package Version 1.0.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.dal; version="[1.0,2.0)"}

```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.dal; version="[1.0,1.1)"}

```

141.6.1 Summary

- **Device** - Represents the device in the OSGi service registry.
- **DeviceException** - {@code DeviceException} is a special {@code IOException}, which is thrown to indicate that there is a device operation fail.
- **DevicePermission** - A bundle's authority to perform specific privileged administrative operations on the devices.
- **Function** - Function service provides specific device operations and properties.
- **FunctionData** - Abstract {@code Function} data wrapper.
- **FunctionEvent** - Asynchronous event, which marks a function property value modification.
- **OperationMetadata** - Contains metadata about function operation.
- **PropertyMetadata** - Contains metadata about a function property, a function operation parameter or a function operation return value.
- **SIUnits** - Contains most of the International System of Units unit symbols.

141.6.2 public abstract Device

Represents the device in the OSGi service registry. Note that {@code Device} services are registered last. Before their registration, there is {@code Function} services registration. The reverse order is used when the services are unregistered. {@code Device} services are unregistered first before {@code Function} services.

141.6.2.1 public static final String DEVICE_CATEGORY = "DAL"

Constant for the value of the {@link Constants#DEVICE_CATEGORY} service property. That category is used by all device services.

See Also Constants.DEVICE_CATEGORY

141.6.2.2 public static final String SERVICE_DESCRIPTION = "dal.device.description"

The service property value contains the device description. It's an optional property. The value type is {@code java.lang.String}.

141.6.2.3 public static final String SERVICE_DRIVER = "dal.device.driver"

The service property value contains the device driver name. For example, EnOcean, Z-Wave, Bluetooth, etc. It's a mandatory property. The value type is {@code java.lang.String}.

141.6.2.4 public static final String SERVICE_FIRMWARE_VENDOR = "dal.device.firmware.vendor"

The service property value contains the device firmware vendor. It's an optional property. The value type is {@code java.lang.String}.

141.6.2.5 public static final String SERVICE_FIRMWARE_VERSION = "dal.device.firmware.version"

The service property value contains the device firmware version. It's an optional property. The value type is {@code java.lang.String}.

141.6.2.6 public static final String SERVICE_HARDWARE_VENDOR = "dal.device.hardware.vendor"

The service property value contains the device hardware vendor. It's an optional property. The value type is {@code java.lang.String}.

141.6.2.7 public static final String SERVICE_HARDWARE_VERSION = "dal.device.hardware.version"

The service property value contains the device hardware version. It's an optional property. The value type is {@code java.lang.String}.

141.6.2.8 public static final String SERVICE_MODEL = "dal.device.model"

The service property value contains the device model. It's an optional property. The value type is {@code java.lang.String}.

141.6.2.9 public static final String SERVICE_NAME = "dal.device.name"

The service property value contains the device name. It's an optional property. The value type is {@code java.lang.String}.

141.6.2.10 public static final String SERVICE_REFERENCE_UIDS = "dal.device.reference.UIDS"

The service property value contains the reference device unique identifiers. It's an optional property. The value type is {@code java.lang.String[]}. It can be used to represent different relationships between the devices. For example, the EnOcean controller can have a reference to the USB dongle.

141.6.2.11 public static final String SERVICE_SERIAL_NUMBER = "dal.device.serial.number"

The service property value contains the device serial number. It's an optional property. The value type is {@code java.lang.String}.

141.6.2.12 public static final String SERVICE_STATUS = "dal.device.status"

The service property value contains the device status. It's a mandatory property. The value type is {@code java.lang.Integer}. The possible values are:

- {@link #STATUS_ONLINE}
- {@link #STATUS_OFFLINE}
- {@link #STATUS_REMOVED}
- {@link #STATUS_PROCESSING}
- {@link #STATUS_NOT_INITIALIZED}
- {@link #STATUS_NOT_CONFIGURED}

141.6.2.13 public static final String SERVICE_STATUS_DETAIL = "dal.device.status.detail"

The service property value contains the device status detail. It holds the reason for the current device status. It's an optional property. The value type is {@code java.lang.Integer}. There are two value categories:

- positive values - those values contain details related to the current status. Examples: {@link #STATUS_DETAIL_CONNECTING} and {@link #STATUS_DETAIL_INITIALIZING}.
- negative values - those values contain errors related to the current status. Examples: {@link #STATUS_DETAIL_CONFIGURATION_UNAPPLIED}, {@link #STATUS_DETAIL_BROKEN} and {@link #STATUS_DETAIL_COMMUNICATION_ERROR}.

141.6.2.14 public static final String SERVICE_TYPES = "dal.device.types"

The service property value contains the device types like DVD, TV, etc. It's an optional property. The value type is {@code java.lang.String[]}.

141.6.2.15 public static final String SERVICE_UID = "dal.device.UID"

The service property value contains the device unique identifier. It's a mandatory property. The value type is {@code java.lang.String}. To simplify the unique identifier generation, the property value must follow the rule:

UID ::= driver-name ':' device-id

UID - device unique identifier

driver-name - the value of the {@link #SERVICE_DRIVER} service property

device-id - device unique identifier in the scope of the driver

141.6.2.16 public static final Integer STATUS_DETAIL_BROKEN

Device status detail indicates that the device is broken. It can be used as a value of {@link #SERVICE_STATUS_DETAIL} service property. The device status must be {@link #STATUS_OFFLINE}.

141.6.2.17 public static final Integer STATUS_DETAIL_COMMUNICATION_ERROR

Device status detail indicates that the device communication is problematic. It can be used as a value of {@link #SERVICE_STATUS_DETAIL} service property. The device status must be {@link #STATUS_ONLINE} or {@link #STATUS_NOT_INITIALIZED}.

141.6.2.18 public static final Integer STATUS_DETAIL_CONFIGURATION_ERROR

Device status detail indicates that the device cannot be configured. It can be used as a value of {@link #SERVICE_STATUS_DETAIL} service property. The device status must be {@link #STATUS_NOT_CONFIGURED}.

- 141.6.2.19** **public static final Integer STATUS_DETAIL_CONFIGURATION_UNAPPLIED**
- Device status detail indicates that the device configuration is not applied. It can be used as a value of {@link #SERVICE_STATUS_DETAIL} service property. The device status must be {@link #STATUS_NOT_CONFIGURED}.
- 141.6.2.20** **public static final Integer STATUS_DETAIL_CONNECTING**
- Device status detail indicates that the device is currently connecting to the network. It can be used as a value of {@link #SERVICE_STATUS_DETAIL} service property. The device status must be {@link #STATUS_PROCESSING}.
- 141.6.2.21** **public static final Integer STATUS_DETAIL_DATA_INSUFFICIENT**
- Device status detail indicates that the device doesn't provide enough information and cannot be determined. It can be used as a value of {@link #SERVICE_STATUS_DETAIL} service property. The device status must be {@link #STATUS_NOT_INITIALIZED}.
- 141.6.2.22** **public static final Integer STATUS_DETAIL_DUTY_CYCLE**
- Device status detail indicates that the device is in duty cycle. It can be used as a value of {@link #SERVICE_STATUS_DETAIL} service property. The device status must be {@link #STATUS_OFFLINE}.
- 141.6.2.23** **public static final Integer STATUS_DETAIL_FIRMWARE_UPDATING**
- Device status detail indicates that the device firmware is updating. It can be used as a value of {@link #SERVICE_STATUS_DETAIL} service property. The device status must be {@link #STATUS_PROCESSING}.
- 141.6.2.24** **public static final Integer STATUS_DETAIL_INACCESSIBLE**
- Device status detail indicates that the device is not accessible and further communication is not possible. It can be used as a value of {@link #SERVICE_STATUS_DETAIL} service property. The device status must be {@link #STATUS_OFFLINE}.
- 141.6.2.25** **public static final Integer STATUS_DETAIL_INITIALIZING**
- Device status detail indicates that the device is currently in process of initialization. It can be used as a value of {@link #SERVICE_STATUS_DETAIL} service property. The device status must be {@link #STATUS_PROCESSING}.
- 141.6.2.26** **public static final Integer STATUS_DETAIL_REMOVING**
- Device status detail indicates that the device is leaving the network. It can be used as a value of {@link #SERVICE_STATUS_DETAIL} service property. The device status must be {@link #STATUS_PROCESSING}.
- 141.6.2.27** **public static final Integer STATUS_NOT_CONFIGURED**
- Device status indicates that the device is currently not configured. The device can require additional actions to become completely connected to the network. It can be used as a value of {@link #SERVICE_STATUS} service property.
- 141.6.2.28** **public static final Integer STATUS_NOT_INITIALIZED**
- Device status indicates that the device is currently not initialized. Some protocols don't provide device information right after the device is connected. The device can be initialized later when it's awakened. It can be used as a value of {@link #SERVICE_STATUS} service property.
- 141.6.2.29** **public static final Integer STATUS_OFFLINE**
- Device status indicates that the device is currently not available for operations. It can be used as a value of {@link #SERVICE_STATUS} service property.

141.6.2.30 public static final Integer STATUS_ONLINE

Device status indicates that the device is currently available for operations. The recent communication with the device has been passed through. It can be used as a value of {@link #SERVICE_STATUS} service property.

141.6.2.31 public static final Integer STATUS_PROCESSING

Device status indicates that the device is currently busy with an operation. It can be used as a value of {@link #SERVICE_STATUS} service property.

141.6.2.32 public static final Integer STATUS_REMOVED

Device status indicates that the device has been removed from the network. That status must be set as the last device status. After that the device service can be unregistered from the service registry. It can be used as a value of {@link #SERVICE_STATUS} service property.

141.6.2.33 public abstract Object getServiceProperty(String propKey)

propKey The property key.

- Returns the current value of the specified property. The method will return the same value as {@code ServiceReference.getProperty(String)} for the service reference of this device.

This method must continue to return property values after the device service has been unregistered.

Returns The property value or {@code null} if the property key cannot be mapped to a value.

141.6.2.34 public abstract String[] getServicePropertyKeys()

- Returns an array with all device service property keys. The method will return the same value as {@code ServiceReference.getPropertyKeys()} for the service reference of this device. The result cannot be {@code null}.

Returns An array with all device service property keys, cannot be {@code null}.

141.6.2.35 public abstract void remove() throws DeviceException

- Removes this device.

The method must synchronously:

- Remove the device from the device network.
- Set the device status to {@link #STATUS_REMOVED}.
- Unregister the device service from the OSGi service registry.

The caller should release the device service after successful execution, because the device will not be operational.

Throws DeviceException – If an operation error is available.

UnsupportedOperationException – If the operation is not supported over this device.

SecurityException – If the caller does not have the appropriate {@code DevicePermission(this device, {@link DevicePermission#REMOVE})} and the Java Runtime Environment supports permissions.

IllegalStateException – If this device service object has already been unregistered.

141.6.3 public class DeviceException extends IOException

{@code DeviceException} is a special {@code IOException}, which is thrown to indicate that there is a device operation fail. The error reason can be located with {@link #getCode()} method. The cause is available with {@link #getCause()}.

141.6.3.1 public static final int COMMUNICATION_ERROR = 1

An exception code indicates that there is an error in the communication.

141.6.3.2 public static final int NO_DATA = 4

An exception code indicates that the requested value is currently not available.

141.6.3.3 public static final int NOT_INITIALIZED = 3

An exception code indicates that the device is not initialized. The device status is {@link Device#STATUS_NOT_INITIALIZED} or {@link Device#STATUS_PROCESSING}.

141.6.3.4 public static final int TIMEOUT = 2

An exception code indicates that there is expired timeout without any processing.

141.6.3.5 public static final int UNKNOWN = 0

An exception code indicates that the error is unknown.

141.6.3.6 public DeviceException()

- Construct a new device exception with {@code null} message. The cause is not initialized and the exception code is set to {@link #UNKNOWN}.

141.6.3.7 public DeviceException(String message)

message The exception message.

- Constructs a new device exception with the given message. The cause is not initialized and the exception code is set to {@link #UNKNOWN}.

141.6.3.8 public DeviceException(String message, Throwable cause)

message The exception message.

cause The exception cause.

- Constructs a new device exception with the given message and cause. The exception code is set to {@link #UNKNOWN}.

141.6.3.9 public DeviceException(String message, Throwable cause, int code)

message The exception message.

cause The exception cause.

code The exception code.

- Constructs a new device exception with the given message, cause and code.

141.6.3.10 public int getCode()

- Returns the exception code. It indicates the reason for this exception. The code can be:

- {@link #UNKNOWN}
- {@link #COMMUNICATION_ERROR}
- {@link #TIMEOUT}
- {@link #NOT_INITIALIZED}
- {@link #NO_DATA}
- custom code

Zero and positive values are reserved for this definition and further extensions of the device exception codes. Custom codes can be used only as negative values to prevent potential collisions.

Returns An exception code.

141.6.4 **public class DevicePermission extends BasicPermission**

A bundle's authority to perform specific privileged administrative operations on the devices. The method {@link Device#remove()} is protected with {@link #REMOVE} permission action.

The name of the permission is a filter based. See OSGi Core Specification, Filter Based Permissions. The filter gives an access to all device service properties. Filter attribute names are processed in a case sensitive manner.

141.6.4.1 **public static final String REMOVE = "remove"**

A permission action to remove the device.

141.6.4.2 **public DevicePermission(String filter, String action)**

filter A filter expression that can use any device service property. The filter attribute names are processed in a case insensitive manner. A special value of "*" can be used to match all devices.

action {@link #REMOVE} action.

- Creates a new {@code DevicePermission} with the given filter and actions. The constructor must only be used to create a permission that is going to be checked.

A filter example: (dal.device.hardware.vendor=acme)

An action: remove

Throws `IllegalArgumentException`— If the filter syntax is not correct or invalid action is specified.

`NullPointerException`— If the filter or action is null.

141.6.4.3 **public DevicePermission(Device device, String action)**

device The device that needs to be checked for a permission.

action {@link #REMOVE} action.

- Creates a new {@code DevicePermission} with the given device and actions. The permission must be used for the security checks like:

{@code securityManager.checkPermission(new DevicePermission(this, "remove"))}. The permissions constructed by this constructor must not be added to the {@code DevicePermission} permission collections.

Throws `IllegalArgumentException`— If an invalid action is specified.

`NullPointerException`— If the device or action is null.

141.6.4.4 **public boolean equals(Object obj)**

obj The object being compared for equality with this object.

- Two {@code DevicePermission} instances are equal if:
 - Represents the same filter and action.
 - Represents the same device (in respect to device unique identifier) and action.

Returns {@code true} if two permissions are equal, {@code false} otherwise.

141.6.4.5 **public String getActions()**

- Returns the canonical string representation of {@link #REMOVE} action.

Returns The canonical string representation of the actions.

141.6.4.6 public int hashCode()

- Returns the hash code value for this object.

Returns Hash code value for this object.

141.6.4.7 public boolean implies(Permission p)

- p* The permission to be implied. It must be constructed by {@link #DevicePermission(Device, String)}.
- Determines if the specified permission is implied by this object. The method will return {@code false} if the specified permission was not constructed by {@link #DevicePermission(Device, String)}. Returns {@code true} if the specified permission is a {@code DevicePermission} and this permission filter matches the specified permission device properties.

Returns {@code true} if the specified permission is implied by this permission, {@code false} otherwise.

Throws IllegalArgumentException – If the specified permission is not constructed by {@link #DevicePermission(Device, String)}.

141.6.4.8 public PermissionCollection newPermissionCollection()

- Returns a new {@code PermissionCollection} suitable for storing {@code DevicePermission} instances.

Returns A new {@code PermissionCollection} instance.

141.6.5 public abstract Function

Function service provides specific device operations and properties. Each function service must implement this interface. In addition to this interface, the implementation can provide own:

- properties;
- operations.

The function service is registered in the service registry with these service properties:

- {@link #SERVICE_UID} - mandatory service property. The property value contains the function unique identifier.
- {@link #SERVICE_DEVICE_UID} - optional service property. The property value is the Functional Device identifiers. The function belongs to those devices.
- {@link #SERVICE_REFERENCE_UIDS} - optional service property. The property value contains the reference function unique identifiers.
- {@link #SERVICE_TYPE} - mandatory service property. The property value is the function type.
- {@link #SERVICE_VERSION} - optional service property. The property value contains the function version.
- {@link #SERVICE_DESCRIPTION} - optional service property. The property value is the function description.
- {@link #SERVICE_OPERATION_NAMES} - optional service property. The property is missing when there are no function operations and property must be set when there are function operations. The property value is the function operation names.
- {@link #SERVICE_PROPERTY_NAMES} - optional service property. The property is missing when there are no function properties and property must be set when there are function properties. The property value is the function property names.

On start up, the {@code Function} services are registered before the {@code Device} services. It's possible that {@link #SERVICE_DEVICE_UID} point to missing services at the moment of the registration. The reverse order is used when the services are unregistered. {@code Function} services are unregistered last after {@code Device} services.

The {@code Function} service should be registered only under the function class hierarchy. Other classes can be used if there are no ambiguous representations. For example, an ambiguous representation can be a function registered under two independent function classes like {@code BinarySwitch} and {@code Meter}. In this example, both functions support the same property {@code state} with different meaning. {@code getPropertyMetadata(String propertyName)} method cannot determinate which property is requested. It can be `BinarySwitch` state or `Meter` state.

To simplify the generic function discovery, the {@code Function} interface must be used for the service registration. In this way, the generic applications can easily find all services, which are functions in the service registry. Because of this rule, this registration is not allowed:

```
context.registerService(MeterV1.class.getName(), this, regProps);
```

If the implementation would like to mark that there is a function, but no specific function interface exists, the registration can be:

```
context.registerService(Function.class.getName(), this, regProps);
```

Note that such functions usually don't have operations and properties.

The function properties must be integrated according to these rules:

- Getter methods must be available for all properties with {@link PropertyMetadata#ACCESS_READABLE} access.
- Getter method must return a subclass of {@link FunctionData}.
- Setter methods must be available for all properties with {@link PropertyMetadata#ACCESS_WRITABLE} access.
- Setter methods can be any combination of:
 - Setter method which accepts a subclass of {@link FunctionData}.
 - Setter method which accepts the values used by the {@link FunctionData} subclass, if there are no equal types.

It's possible to have only one or both of them. Examples:

- There is {@code MyFunctionData} bean with {@code BigDecimal} value for a {@code data} property. Valid setters are {@code setData(MyFunctionData data)} and {@code setData(BigDecimal data)}.
- There is {@code MySecondFunctionData} bean with {@code BigDecimal} prefix and {@code BigDecimal} suffix for a {@code data} property. The prefix and suffix are using equal types and we cannot have a setter with the values used by {@code MySecondFunctionData}. The only one possible setter is {@code setData(MySecondFunctionData data)}.
- No methods are required for properties with {@link PropertyMetadata#ACCESS_EVENTABLE} access.

The accessor method names must be defined according JavaBeans specification.

The function operations are java methods, which cannot override the property accessor methods. They can have zero or more parameters and zero or one return value.

Operation arguments and function properties are restricted by the same set of rules. The data type can be one of the following types:

- Java primitive type or corresponding reference type.
- {@code java.lang.String}.
- Numerical type i.e. the type which extends `java.lang.Number`. The numerical type must follow these conventions:
 - The type must provide a public static method called {@code valueOf} that returns an instance of the given type and takes a single {@code String} argument or a public constructor which takes a single {@code String} argument.

- The {@code String} argument from the previous bullet can be provided by {@code toString()} method of the instance.
- {@code Beans}, but the beans properties must use those rules. Java Beans are defined in JavaBeans specification.
- {@code java.util.Map}s. The keys can be {@code java.lang.String}. The values of a single type follow these rules.
- Arrays of defined types.

The properties metadata is accessible with {@link #getPropertyMetadata(String)}. The operations metadata is accessible with {@link #getOperationMetadata(String)}.

In order to provide common behavior, all functions must follow a set of common rules related to the implementation of their setters, getters, operations and events:

- The setter method must be executed synchronously. If the underlying protocol can return response to the setter call, it must be awaited. It simplifies the property value modifications and doesn't require asynchronous callback.
- The operation method must be executed synchronously. If the underlying protocol can return an operation confirmation or response, they must be awaited. It simplifies the operation execution and doesn't require asynchronous callback.
- The getter must return the last known cached property value. The device implementation is responsible to keep that value up to date. It'll speed up the applications when the function property values are collected. The same cached value can be shared between a few requests instead of a few calls to the real device.
- The function operations, getters and setters must not override {@code java.lang.Object} and this interface methods.

141.6.5.1 **public static final String SERVICE_DESCRIPTION = "dal.function.description"**

The service property value contains the function description. It's an optional property. The value type is {@code java.lang.String}.

141.6.5.2 **public static final String SERVICE_DEVICE_UID = "dal.function.device.UID"**

The service property value contains the device unique identifier. The function belongs to this device. It's an optional property. The value type is {@code java.lang.String}.

141.6.5.3 **public static final String SERVICE_OPERATION_NAMES = "dal.function.operation.names"**

The service property value contains the function operation names. It's an optional property. The property is missing when there are no function operations and property must be set when there are function operations. The value type is {@code java.lang.String[]}. It's not possible to exist two or more function operations with the same name i.e. the operation overloading is not allowed.

141.6.5.4 **public static final String SERVICE_PROPERTY_NAMES = "dal.function.property.names"**

The service property value contains the function property names. It's an optional property. The property is missing when there are no function properties and property must be set when there are function properties. The value type is {@code java.lang.String[]}. It's not possible to exist two or more function properties with the same name.

141.6.5.5 **public static final String SERVICE_REFERENCE_UIDS = "dal.function.reference.UIDs"**

The service property value contains the reference function unique identifiers. It's an optional property. The value type is {@code java.lang.String[]}. It can be used to represent different relationships between the functions.

141.6.5.6 public static final String SERVICE_TYPE = "dal.function.type"

The service property value contains the function type. It's an optional property. For example, the sensor function can have different types like temperature, pressure, etc. The value type is {@code java.lang.String}.

Organizations that want to use function types that do not clash with OSGi Working Group defined types should prefix their types in own namespace.

The type doesn't mandate specific function interface. It can be used with different functions.

141.6.5.7 public static final String SERVICE_UID = "dal.function.UID"

The service property value contains the function unique identifier. It's a mandatory property. The value type is {@code java.lang.String}. To simplify the unique identifier generation, the property value must follow the rule:

function UID ::= device-id ':' function-id

function UID - function unique identifier

device-id - the value of the {@link Device#SERVICE_UID} Device service property

function-id - function identifier in the scope of the device

If the function is not bound to a device, the function unique identifier can be device independent.

141.6.5.8 public static final String SERVICE_VERSION = "dal.function.version"

The service property value contains the function version. That version can point to specific implementation version and vary in the different vendor implementations. It's an optional property. The value type is {@code java.lang.String}.

141.6.5.9 public abstract OperationMetadata getOperationMetadata(String operationName)

operationName The function operation name, for which metadata is requested.

- Provides metadata about the function operation.

This method must continue to return the operation metadata after the function service has been unregistered.

Returns The operation metadata for the given operation name. {@code null} if the operation metadata is not available.

Throws `IllegalArgumentException` – If the function operation with the specified name is not available.

141.6.5.10 public abstract PropertyMetadata getPropertyMetadata(String propertyName)

propertyName The function property name, for which metadata is requested.

- Provides metadata about the function property.

This method must continue to return the property metadata after the function service has been unregistered.

Returns The property metadata for the given property name. {@code null} if the property metadata is not available.

Throws `IllegalArgumentException` – If the function property with the specified name is not available.

141.6.5.11 public abstract Object getServiceProperty(String propKey)

propKey The property key.

- Returns the current value of the specified property. The method will return the same value as {@code ServiceReference.getProperty(String)} for the service reference of this function.

This method must continue to return property values after the device function service has been un-registered.

Returns The property value or `{@code null}` if the property key cannot be mapped to a value.

141.6.5.12 **public abstract String[][] getServicePropertyKeys()**

- Returns an array with all function service property keys. The method will return the same value as `{@code ServiceReference.getPropertyKeys()}` for the service reference of this function. The result cannot be `{@code null}`.

Returns An array with all function service property keys, cannot be `{@code null}`.

141.6.6 **public abstract class FunctionData implements Comparable<Object>**

Abstract `{@code Function}` data wrapper. A subclass must be used for an access to the property values by all functions. It takes care about the timestamp and additional metadata. The subclasses are responsible to provide concrete value and unit if required.

141.6.6.1 **public static final String DESCRIPTION = "description"**

Metadata key, which value represents the data description. The property value type is `{@code java.lang.String}`.

141.6.6.2 **public static final String FIELD_METADATA = "metadata"**

Represents the metadata field name. The field value is available with `{@link #getMetadata()}`. The field type is `{@code Map}`. The constant can be used as a key to `{@link #FunctionData(Map)}`.

141.6.6.3 **public static final String FIELD_TIMESTAMP = "timestamp"**

Represents the timestamp field name. The field value is available with `{@link #getTimestamp()}`. The field type is `{@code long}`. The constant can be used as a key to `{@link #FunctionData(Map)}`.

141.6.6.4 **public FunctionData(Map<String, ?> fields)**

fields Contains the new `{@code FunctionData}` instance field values.

- Constructs new `{@code FunctionData}` instance with the specified field values. The map keys must match to the field names. The map values will be assigned to the appropriate class fields. For example, the maps can be: `{"timestamp"=Long(1384440775495)}`. That map will initialize the `{@link #FIELD_TIMESTAMP}` field with 1384440775495. If timestamp is missing, `{@link Long#MIN_VALUE}` is used.
- `{@link #FIELD_TIMESTAMP}` - optional field. The value type must be `{@code Long}`.
- `{@link #FIELD_METADATA}` - optional field. The value type must be `{@code Map}`.

Throws `ClassCastException` – If the field value types are not expected.

`NullPointerException` – If the fields map is `{@code null}`.

141.6.6.5 **public FunctionData(long timestamp, Map<String, ?> metadata)**

timestamp The data timestamp optional field.

metadata The data metadata optional field.

- Constructs new `{@code FunctionData}` instance with the specified arguments.

141.6.6.6 **public int compareTo(Object o)**

o `{@code FunctionData}` to be compared.

- Compares this {@code FunctionData} instance with the given argument. If the argument is not {@code FunctionData}, it throws {@code ClassCastException}. Otherwise, this method returns:
 - {@code -1} if this instance timestamp is less than the argument timestamp. If they are equivalent, it can be the result of the metadata map deep comparison.
 - {@code 0} if all fields are equivalent.
 - {@code 1} if this instance timestamp is greater than the argument timestamp. If they are equivalent, it can be the result of the metadata map deep comparison.

Metadata map deep comparison compares the elements of all nested {@code java.util.Map} and array instances. {@code null} is less than any other non-null instance.

Returns {@code -1}, {@code 0} or {@code 1} depending on the comparison rules.

Throws `ClassCastException`— If the method argument is not of type {@code FunctionData} or metadata maps contain values of different types for the same key.

`NullPointerException`— If the method argument is {@code null}.

See Also `java.lang.Comparable.compareTo(java.lang.Object)`

141.6.6.7 **public boolean equals(Object other)**

other The other instance to compare. It must be of {@code FunctionData} type.

- Two {@code FunctionData} instances are equal if their metadata and timestamp are equivalent.

Returns {@code true} if this instance and argument have equivalent metadata and timestamp, {@code false} otherwise.

See Also `java.lang.Object.equals(java.lang.Object)`

141.6.6.8 **public Map<String, ?> getMetadata()**

- Returns {@code FunctionData} metadata. It's dynamic metadata related only to this specific value. Possible keys:
 - {@link #DESCRIPTION}
 - custom key

Returns {@code FunctionData} metadata or {@code null} if there is no metadata.

141.6.6.9 **public long getTimestamp()**

- Returns {@code FunctionData} timestamp. The timestamp is the difference between the value collecting time and midnight, January 1, 1970 UTC. It's measured in milliseconds. The device driver is responsible to generate that value when the value is received from the device. {@link java.lang.Long#MIN_VALUE} value means no timestamp.

Returns {@code FunctionData} timestamp.

141.6.6.10 **public int hashCode()**

- Returns the hash code of this {@code FunctionData}.

Returns {@code FunctionData} hash code.

See Also `java.lang.Object.hashCode()`

141.6.7 **public class FunctionEvent extends Event**

Asynchronous event, which marks a function property value modification. The event can be triggered when there is a new property value, but it's possible to have events in series with no value change. The event properties must contain:

- {@link #FUNCTION_UID} - the event source function unique identifier.
- {@link #PROPERTY_NAME} - the property name.
- {@link #PROPERTY_VALUE} - the property value. The property value type must be a subclass of `FunctionData`.

141.6.7.1 **public static final String EVENT_CLASS = "org/osgi/service/dal/FunctionEvent/"**

Represents the event class. That constant can be useful for the event handlers depending on the event filters.

141.6.7.2 **public static final String EVENT_PACKAGE = "org/osgi/service/dal/"**

Represents the event package. That constant can be useful for the event handlers depending on the event filters.

141.6.7.3 **public static final String FUNCTION_UID = "dal.function.UID"**

Represents an event property key for function UID. The property value type is {@code java.lang.String}. The value represents the property value change source function identifier.

141.6.7.4 **public static final String PROPERTY_NAME = "dal.function.property.name"**

Represents an event property key for the function property name. The property value type is {@code java.lang.String}. The value represents the property name.

141.6.7.5 **public static final String PROPERTY_VALUE = "dal.function.property.value"**

Represents an event property key for the function property value. The property value type is a subclass of {@code FunctionData}. The value represents the property value.

141.6.7.6 **public static final String TOPIC_PROPERTY_CHANGED = "org/osgi/service/dal/FunctionEvent/PROPERTY_CHANGED"**

Represents the event topic for the function property changed.

141.6.7.7 **public FunctionEvent(String topic, Dictionary<String, ?> properties)**

topic The event topic.

properties The event properties.

- Constructs a new event with the specified topic and properties.

141.6.7.8 **public FunctionEvent(String topic, Map<String, ?> properties)**

topic The event topic.

properties The event properties.

- Constructs a new event with the specified topic and properties.

141.6.7.9 **public FunctionEvent(String topic, String functionUID, String propName, FunctionData propValue)**

topic The event topic.

functionUID The event source function UID.

propName The event source property name.

propValue The event source property value.

- Constructs a new event with the specified topic, function UID, property name and property value.

141.6.7.10 **public String getFunctionPropertyName()**

- Returns the property name. The value is same as the value of {@link #PROPERTY_NAME}.

Returns The property name.

141.6.7.11 public FunctionData getFunctionPropertyValue()

- Returns the property value. The value is same as the value of {@link #PROPERTY_VALUE}.

Returns The property value.

141.6.7.12 public String getFunctionUID()

- Returns the property value change source function identifier. The value is same as the value of {@link #FUNCTION_UID} property.

Returns The property value change source function.

141.6.8 public abstract OperationMetadata

Contains metadata about function operation.

See Also Function , PropertyMetadata

141.6.8.1 public static final String DESCRIPTION = "description"

Metadata key, which value represents the operation description. The property value type is {@code java.lang.String}.

141.6.8.2 public abstract Map<String, ?> getMetadata()

- Returns metadata about the function operation. The keys of the {@code java.util.Map} result must be of {@code java.lang.String} type. Possible keys:
 - {@link #DESCRIPTION}
 - custom key

Returns The operation metadata or {@code null} if no such metadata is available.

141.6.8.3 public abstract PropertyMetadata[][] getParametersMetadata()

- Returns metadata about the operation parameters or {@code null} if no such metadata is available.

Returns Operation parameters metadata.

141.6.8.4 public abstract PropertyMetadata getReturnValueMetadata()

- Returns metadata about the operation return value or {@code null} if no such metadata is available.

Returns Operation return value metadata.

141.6.9 public abstract PropertyMetadata

Contains metadata about a function property, a function operation parameter or a function operation return value. The access to the function properties is a bitmap value of {@link #ACCESS} metadata key. Function properties can be accessed in three ways. Any combinations between them are possible:

- {@link #ACCESS_READABLE} - available for all properties, which can be read. Function must provide a getter method for an access to the property value.
- {@link #ACCESS_WRITABLE} - available for all properties, which can be modified. Function must provide a setter method for a modification of the property value.
- {@link #ACCESS_EVENTABLE} - available for all properties, which can report the property value. {@link FunctionEvent}s are sent on property change.

See Also Function , PropertyMetadata

141.6.9.1 public static final String ACCESS = "access"

Metadata key, which value represents the access to the function property. The property value is a bitmap of {@code Integer} type. The bitmap can be any combination of:

- {@link #ACCESS_READABLE}
- {@link #ACCESS_WRITABLE}
- {@link #ACCESS_EVENTABLE}

For example, value {@code Integer(3)} means that the property is readable and writable, but not eventable.

The property access is available only for function properties and it's missing for the operation parameters.

141.6.9.2 public static final int ACCESS_EVENTABLE = 4

Marks the eventable function properties. The flag can be used as a part of bitmap value of {@link #ACCESS}.

See Also Function

141.6.9.3 public static final int ACCESS_READABLE = 1

Marks the readable function properties. The flag can be used as a part of bitmap value of {@link #ACCESS}. The readable access mandates function to provide a property getter method.

See Also Function

141.6.9.4 public static final int ACCESS_WRITABLE = 2

Marks the writable function properties. The flag can be used as a part of bitmap value of {@link #ACCESS}. The writable access mandates function to provide a property setter methods.

See Also Function

141.6.9.5 public static final String DESCRIPTION = "description"

Metadata key, which value represents the property description. The property value type is {@code java.lang.String}.

141.6.9.6 public static final String UNITS = "units"

Metadata key, which value represents the property supported units. The property value type is {@code java.lang.String[]}. The array first element at index {@code 0} represents the default unit. Each unit must follow those rules:

- The International System of Units must be used where it's applicable. For example, kg for kilogram and km for kilometer.
- If the unit name matches to an Unicode symbol name, the Unicode symbol must be used. For example, the degree unit matches to the Unicode degree sign ({@code "°"}).
- If the unit name doesn't match to an Unicode symbol, the unit symbol must be built by Unicode Basic Latin block of characters, superscript and subscript characters. For example, watt per square meter steradian is built by {@code W/(m².sr)}.

If those rules cannot be applied to the unit symbol, custom rules are allowed. A set of predefined unit symbols are available in {@link SIUnits} interface.

141.6.9.7 public abstract FunctionData[][] getEnumValues(String unit)

unit The unit to align the supported values, can be {@code null}.

- Returns the property possible values according to the specified unit. If the unit is {@code null}, the values set is aligned to the default unit. If there is no such set of supported values, {@code null} is returned. The values must be sorted in increasing order.

Returns The supported values according to the specified unit or {@code null} if no such values are supported. The values must be sorted in increasing order.

Throws `IllegalArgumentException` – If the unit is not supported.

141.6.9.8 **public abstract FunctionData getMaxValue(String unit)**

unit The unit to align the maximum value, can be {@code null}.

- Returns the property maximum value according to the specified unit. If the unit is {@code null}, the maximum value is aligned to the default unit. If there is no maximum value, {@code null} is returned.

Returns The maximum value according to the specified unit or {@code null} if no maximum value is supported.

Throws `IllegalArgumentException` – If the unit is not supported.

141.6.9.9 **public abstract Map<String, ?> getMetadata(String unit)**

unit The unit to align the metadata if it's applicable. It can be null, which means that the default unit will be used.

- Returns metadata about the function property or operation parameter. The keys of the {@code java.util.Map} result must be of {@code java.lang.String} type. Possible keys:
 - {@link #DESCRIPTION} - doesn't depend on the given unit.
 - {@link #ACCESS} - available only for function property and missing for function operation parameters. It doesn't depend on the given unit.
 - {@link #UNITS} - doesn't depend on the given unit.
 - custom key - can depend on the unit. Organizations that want to use custom keys that do not clash with OSGi Working Group defined should prefix their keys in own namespace.

Returns The property metadata or {@code null} if no such metadata is available.

141.6.9.10 **public abstract FunctionData getMinValue(String unit)**

unit The unit to align the minimum value, can be {@code null}.

- Returns the property minimum value according to the specified unit. If the unit is {@code null}, the minimum value is aligned to the default unit. If there is no minimum value, {@code null} is returned.

Returns The minimum value according to the specified unit or {@code null} if no minimum value is supported.

Throws `IllegalArgumentException` – If the unit is not supported.

141.6.9.11 **public abstract FunctionData getStep(String unit)**

unit The unit to align the step, can be {@code null}.

- Returns the difference between two values in series. For example, if the range is [0, 100], the step can be 10.

Returns The step according to the specified unit or {@code null} if no step is supported.

Throws `IllegalArgumentException` – If the unit is not supported.

141.6.10 public final class SIUnits

Contains most of the International System of Units unit symbols. The constant name represents the unit name. The constant value represents the unit symbol as it's defined in {@link PropertyMetadata#UNITS}.

141.6.10.1 public static final String AMPERE = "A"

Unit of electric current defined by the International System of Units (SI). It's one of the base units called ampere.

141.6.10.2 public static final String AMPERE_PER_METER = "A/m"

Unit of magnetic field strength. It's one of the coherent derived units in the SI expressed in terms of base units. The unit is called ampere per meter.

141.6.10.3 public static final String AMPERE_PER_SQUARE_METER = "A/m²"

Unit of current density. It's one of the coherent derived units in the SI expressed in terms of base units. The unit is called ampere per square meter.

141.6.10.4 public static final String ANGSTROM = "Å"

Unit of length. It's one of the other non-SI units. The unit is called angstrom.

141.6.10.5 public static final String BAR = "bar"

Unit of pressure. It's one of the other non-SI units. The unit is called bar.

141.6.10.6 public static final String BARN = "b"

Unit of area. It's one of the other non-SI units. The unit is called barn.

141.6.10.7 public static final String BECQUEREL = "Bq"

Unit of activity referred to a radionuclide. It's one of the coherent derived units in the SI with special names and symbols. The unit is called becquerel.

141.6.10.8 public static final String BEL = "B"

Unit of logarithmic ratio quantities. It's one of the other non-SI units. The unit is called bel.

141.6.10.9 public static final String CANDELA = "cd"

Unit of luminous intensity defined by the International System of Units (SI). It's one of the base units called candela.

141.6.10.10 public static final String CANDELA_PER_SQUARE_METER = "cd/m²"

Unit of luminance. It's one of the coherent derived units in the SI expressed in terms of base units. The unit is called candela per square meter.

141.6.10.11 public static final String COULOMB = "C"

Unit of electronic charge, amount of electricity. It's one of the coherent derived units in the SI with special names and symbols. The unit is called coulomb.

141.6.10.12 public static final String COULOMB_PER_CUBIC_METER = "C/m³"

Unit of electric charge density. It's one of the coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called coulomb per cubic meter.

- 141.6.10.13** **public static final String COULOMB_PER_KILOGRAM = "C/kg"**
Unit of exposure (x- and gamma-rays). It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called coulomb per kilogram.
- 141.6.10.14** **public static final String COULOMB_PER_SQUARE_METER = "C/m²"**
Unit of surface charge density, electric flux density, electric displacement. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called coulomb per square meter.
- 141.6.10.15** **public static final String CUBIC_METER = "m³"**
Unit of volume. It's one of coherent derived units in the SI expressed in terms of base units. The unit is called cubic meter.
- 141.6.10.16** **public static final String CUBIC_METER_PER_KILOGRAM = "m³/kg"**
Unit of specific volume. It's one of coherent derived units in the SI expressed in terms of base units. The unit is called cubic meter per kilogram.
- 141.6.10.17** **public static final String DAY = "d"**
Unit of time. It's one of non-SI units accepted for use with the International System of Units. The unit is called day.
- 141.6.10.18** **public static final String DECIBEL = "dB"**
Unit of logarithmic ratio quantities. It's one of other non-SI units. The unit is called decibel.
- 141.6.10.19** **public static final String DEGREE = "°"**
Unit of plane angle. It's one of non-SI units accepted for use with the International System of Units. The unit is called degree.
- 141.6.10.20** **public static final String DEGREE_CELSIUS = "°C"**
Unit of Celsius temperature. It's one of the coherent derived units in the SI with special names and symbols. The unit is called degree Celsius.
- 141.6.10.21** **public static final String DYNE = "dyn"**
Unit of force. It's one of non-SI units associated with the CGS and the CGS-Gaussian system of units. The unit is called dyne.
- 141.6.10.22** **public static final String ERG = "erg"**
Unit of energy. It's one of non-SI units associated with the CGS and the CGS-Gaussian system of units. The unit is called erg.
- 141.6.10.23** **public static final String FARAD = "F"**
Unit of capacitance. It's one of the coherent derived units in the SI with special names and symbols. The unit is called farad.
- 141.6.10.24** **public static final String FARAD_PER_METER = "F/m"**
Unit of permittivity. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called farad per meter.
- 141.6.10.25** **public static final String GAL = "Gal"**
Unit of acceleration. It's one of non-SI units associated with the CGS and the CGS-Gaussian system of units. The unit is called gal.

141.6.10.26 public static final String GAUSS = "G"

Unit of magnetic flux density. It's one of non-SI units associated with the CGS and the CGS-Gaussian system of units. The unit is called gauss.

141.6.10.27 public static final String GRAY = "Gy"

Unit of absorbed dose, specific energy (imparted), kerma. It's one of the coherent derived units in the SI with special names and symbols. The unit is called gray.

141.6.10.28 public static final String GRAY_PER_SECOND = "Gy/s"

Unit of absorbed dose rate. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called gray per second.

141.6.10.29 public static final String HECTARE = "ha"

Unit of area. It's one of non-SI units accepted for use with the International System of Units. The unit is called hectare.

141.6.10.30 public static final String HENRY = "H"

Unit of inductance. It's one of the coherent derived units in the SI with special names and symbols. The unit is called henry.

141.6.10.31 public static final String HENRY_PER_METER = "H/m"

Unit of permeability. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called henry per meter.

141.6.10.32 public static final String HERTZ = "Hz"

Unit of frequency. It's one of the coherent derived units in the SI with special names and symbols. The unit is called hertz.

141.6.10.33 public static final String HOUR = "h"

Unit of time. It's one of non-SI units accepted for use with the International System of Units. The unit is called hour.

141.6.10.34 public static final String JOULE = "J"

Unit of energy, work, amount of electricity. It's one of the coherent derived units in the SI with special names and symbols. The unit is called joule.

141.6.10.35 public static final String JOULE_PER_CUBIC_METER = "J/m³"

Unit of energy density. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called joule per cubic meter.

141.6.10.36 public static final String JOULE_PER_KELVIN = "J/K"

Unit of heat capacity, entropy. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called joule per kelvin.

141.6.10.37 public static final String JOULE_PER_KILOGRAM = "J/kg"

Unit of specific energy. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called joule per kilogram.

141.6.10.38 public static final String JOULE_PER_KILOGRAM_KELVIN = "J/(kg K)"

Unit of specific heat capacity, specific entropy. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called joule per kilogram kelvin.

- 141.6.10.39** **public static final String JOULE_PER_MOLE = "J/mol"**
Unit of molar energy. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called joule per mole.
- 141.6.10.40** **public static final String JOULE_PER_MOLE_KELVIN = "J/(mol K)"**
Unit of molar entropy, molar heat capacity. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called joule per mole kelvin.
- 141.6.10.41** **public static final String KATAL = "kat"**
Unit of catalytic activity. It's one of the coherent derived units in the SI with special names and symbols. The unit is called katal.
- 141.6.10.42** **public static final String KATAL_PER_CUBIC_METER = "kat/m³"**
Unit of catalytic activity concentration. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called katal per cubic meter.
- 141.6.10.43** **public static final String KELVIN = "K"**
Unit of thermodynamic temperature defined by the International System of Units (SI). It's one of the base units called kelvin.
- 141.6.10.44** **public static final String KILOGRAM = "kg"**
Unit of mass defined by the International System of Units (SI). It's one of the base units called kilogram.
- 141.6.10.45** **public static final String KILOGRAM_PER_CUBIC_METER = "kg/m³"**
Unit of density, mass density, mass concentration. It's one of coherent derived units in the SI expressed in terms of base units. The unit is called kilogram per cubic meter.
- 141.6.10.46** **public static final String KILOGRAM_PER_SQUARE_METER = "kg/m²"**
Unit of surface density. It's one of coherent derived units in the SI expressed in terms of base units. The unit is called kilogram per square meter.
- 141.6.10.47** **public static final String KNOT = "kn"**
Unit of speed. It's one of other non-SI units. The unit is called knot.
- 141.6.10.48** **public static final String LITER = "l"**
Unit of volume. It's one of non-SI units accepted for use with the International System of Units. The unit is called liter. International System of Units accepts two symbols: lower-case l and capital L. That constant value is using the lower-case l.
- 141.6.10.49** **public static final String LUMEN = "lm"**
Unit of luminous flux. It's one of the coherent derived units in the SI with special names and symbols. The unit is called lumen.
- 141.6.10.50** **public static final String LUX = "lx"**
Unit of illuminance. It's one of the coherent derived units in the SI with special names and symbols. The unit is called lux.
- 141.6.10.51** **public static final String MAXWELL = "Mx"**
Unit of magnetic flux. It's one of non-SI units associated with the CGS and the CGS-Gaussian system of units. The unit is called maxwell.

- 141.6.10.52** **public static final String METER = "m"**
Unit of length defined by the International System of Units (SI). It's one of the base units called meter.
- 141.6.10.53** **public static final String METER_PER_SECOND = "m/s"**
Unit of speed, velocity. It's one of the coherent derived units in the SI expressed in terms of base units. The unit is called meter per second.
- 141.6.10.54** **public static final String METER_PER_SECOND_SQUARED = "m/s²"**
Unit of acceleration. It's one of the coherent derived units in the SI expressed in terms of base units. The unit is called meter per second squared.
- 141.6.10.55** **public static final String MILLIMETER_OF_MERCURY = "mmHg"**
Unit of pressure. It's one of the other non-SI units. The unit is called millimeter of mercury.
- 141.6.10.56** **public static final String MOLE = "mol"**
Unit of amount of substance defined by the International System of Units (SI). It's one of the base units called mole.
- 141.6.10.57** **public static final String MOLE_PER_CUBIC_METER = "mol/m³"**
Unit of amount concentration, concentration. It's one of the coherent derived units in the SI expressed in terms of base units. The unit is called mole per cubic meter.
- 141.6.10.58** **public static final String NAUTICAL_MILE = "M"**
Unit of distance. It's one of the other non-SI units. The unit is called nautical mile.
- 141.6.10.59** **public static final String NEPER = "Np"**
Unit of logarithmic ratio quantities. It's one of the other non-SI units. The unit is called neper.
- 141.6.10.60** **public static final String NEWTON = "N"**
Unit of force. It's one of the coherent derived units in the SI with special names and symbols. The unit is called newton.
- 141.6.10.61** **public static final String NEWTON_METER = "N m"**
Unit of moment of force. It's one of the coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called newton meter.
- 141.6.10.62** **public static final String NEWTON_PER_METER = "N/m"**
Unit of surface tension. It's one of the coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called newton per meter.
- 141.6.10.63** **public static final String OERSTED = "Oe"**
Unit of magnetic field. It's one of the non-SI units associated with the CGS and the CGS-Gaussian system of units. The unit is called oersted.
- 141.6.10.64** **public static final String OHM = "Ω"**
Unit of electric resistance. It's one of the coherent derived units in the SI with special names and symbols. The unit is called ohm.
- 141.6.10.65** **public static final String PASCAL = "Pa"**
Unit of pressure, stress. It's one of the coherent derived units in the SI with special names and symbols. The unit is called pascal.

- 141.6.10.66** **public static final String PASCAL_SECOND = "Pa s"**
Unit of dynamic viscosity. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called pascal second.
- 141.6.10.67** **public static final String PHOT = "ph"**
Unit of illuminance. It's one of non-SI units associated with the CGS and the CGS-Gaussian system of units. The unit is called phot.
- 141.6.10.68** **public static final String PLANE_ANGLE_MINUTE = ""**
Unit of plane angle. It's one of non-SI units accepted for use with the International System of Units. The unit is called minute.
- 141.6.10.69** **public static final String PLANE_ANGLE_SECOND = ""**
Unit of plane angle. It's one of non-SI units accepted for use with the International System of Units. The unit is called second.
- 141.6.10.70** **public static final String POISE = "P"**
Unit of dynamic viscosity. It's one of non-SI units associated with the CGS and the CGS-Gaussian system of units. The unit is called poise.
- 141.6.10.71** **public static final String PREFIX_ATTO = "a"**
Adopted prefix symbol to form the symbols of the decimal submultiples of SI units. It's called atto and represents the 18th negative power of ten.
- 141.6.10.72** **public static final String PREFIX_CENTI = "c"**
Adopted prefix symbol to form the symbols of the decimal submultiples of SI units. It's called centi and represents the 2nd negative power of ten.
- 141.6.10.73** **public static final String PREFIX_DECA = "da"**
Adopted prefix symbol to form the symbols of the decimal multiples of SI units. It's called deca and represents the 1st power of ten.
- 141.6.10.74** **public static final String PREFIX_DECI = "d"**
Adopted prefix symbol to form the symbols of the decimal submultiples of SI units. It's called deci and represents the 1st negative power of ten.
- 141.6.10.75** **public static final String PREFIX_EXA = "E"**
Adopted prefix symbol to form the symbols of the decimal multiples of SI units. It's called exa and represents the 18th power of ten.
- 141.6.10.76** **public static final String PREFIX_FEMTO = "f"**
Adopted prefix symbol to form the symbols of the decimal submultiples of SI units. It's called femto and represents the 15th negative power of ten.
- 141.6.10.77** **public static final String PREFIX_GIGA = "G"**
Adopted prefix symbol to form the symbols of the decimal multiples of SI units. It's called giga and represents the 9th power of ten.
- 141.6.10.78** **public static final String PREFIX_HECTO = "h"**
Adopted prefix symbol to form the symbols of the decimal multiples of SI units. It's called hecto and represents the 2nd power of ten.

- 141.6.10.79** **public static final String PREFIX_KILO = "k"**
Adopted prefix symbol to form the symbols of the decimal multiples of SI units. It's called kilo and represents the 3rd power of ten.
- 141.6.10.80** **public static final String PREFIX_MEGA = "M"**
Adopted prefix symbol to form the symbols of the decimal multiples of SI units. It's called mega and represents the 6th power of ten.
- 141.6.10.81** **public static final String PREFIX_MICRO = "μ"**
Adopted prefix symbol to form the symbols of the decimal submultiples of SI units. It's called micro and represents the 6th negative power of ten.
- 141.6.10.82** **public static final String PREFIX_MILLI = "m"**
Adopted prefix symbol to form the symbols of the decimal submultiples of SI units. It's called milli and represents the 3rd negative power of ten.
- 141.6.10.83** **public static final String PREFIX_NANO = "n"**
Adopted prefix symbol to form the symbols of the decimal submultiples of SI units. It's called nano and represents the 9th negative power of ten.
- 141.6.10.84** **public static final String PREFIX_PICO = "p"**
Adopted prefix symbol to form the symbols of the decimal submultiples of SI units. It's called pico and represents the 12th negative power of ten.
- 141.6.10.85** **public static final String PREFIX_YOCTO = "y"**
Adopted prefix symbol to form the symbols of the decimal submultiples of SI units. It's called yocto and represents the 24th negative power of ten.
- 141.6.10.86** **public static final String PREFIX_YOTTA = "Y"**
Adopted prefix symbol to form the symbols of the decimal multiples of SI units. It's called yotta and represents the 24th power of ten.
- 141.6.10.87** **public static final String PREFIX_ZEPTO = "z"**
Adopted prefix symbol to form the symbols of the decimal submultiples of SI units. It's called zepto and represents the 21th negative power of ten.
- 141.6.10.88** **public static final String PREFIX_ZETTA = "Z"**
Adopted prefix symbol to form the symbols of the decimal multiples of SI units. It's called zetta and represents the 21th power of ten.
- 141.6.10.89** **public static final String RADIAN = "rad"**
Unit of plane angle. It's one of the coherent derived units in the SI with special names and symbols. The unit is called radian.
- 141.6.10.90** **public static final String RADIAN_PER_SECOND = "rad/s"**
Unit of angular velocity. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called radian per second.
- 141.6.10.91** **public static final String RADIAN_PER_SECOND_SQUARED = "rad/s²"**
Unit of angular acceleration. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called radian per second squared.

- 141.6.10.92** **public static final String RECIPROCAL_METER = "m⁻¹"**
Unit of wavenumber. It's one of coherent derived units in the SI expressed in terms of base units. The unit is called reciprocal meter.
- 141.6.10.93** **public static final String SECOND = "s"**
Unit of time defined by the International System of Units (SI). It's one of the base units called second.
- 141.6.10.94** **public static final String SIEMENS = "S"**
Unit of electric conductance. It's one of the coherent derived units in the SI with special names and symbols. The unit is called siemens.
- 141.6.10.95** **public static final String SIEVERT = "Sv"**
Unit of dose equivalent, ambient dose equivalent, directional dose equivalent, personal dose equivalent. It's one of the coherent derived units in the SI with special names and symbols. The unit is called sievert.
- 141.6.10.96** **public static final String SQUARE_METER = "m²"**
Unit of area. It's one of coherent derived units in the SI expressed in terms of base units. The unit is called square meter.
- 141.6.10.97** **public static final String STERADIAN = "sr"**
Unit of solid angle. It's one of the coherent derived units in the SI with special names and symbols. The unit is called steradian.
- 141.6.10.98** **public static final String STILB = "sb"**
Unit of luminance. It's one of non-SI units associated with the CGS and the CGS-Gaussian system of units. The unit is called stilb.
- 141.6.10.99** **public static final String STOKES = "St"**
Unit of kinematic viscosity. It's one of non-SI units associated with the CGS and the CGS-Gaussian system of units. The unit is called stokes.
- 141.6.10.100** **public static final String TESLA = "T"**
Unit of magnetic flux density. It's one of the coherent derived units in the SI with special names and symbols. The unit is called tesla.
- 141.6.10.101** **public static final String TIME_MINUTE = "min"**
Unit of time. It's one of non-SI units accepted for use with the International System of Units. The unit is called minute.
- 141.6.10.102** **public static final String TONNE = "t"**
Unit of mass. It's one of non-SI units accepted for use with the International System of Units. The unit is called tonne.
- 141.6.10.103** **public static final String VOLT = "V"**
Unit of electric potential difference, electromotive force. It's one of the coherent derived units in the SI with special names and symbols. The unit is called volt.
- 141.6.10.104** **public static final String VOLT_PER_METER = "V/m"**
Unit of electric field strength. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called volt per meter.

141.6.10.105

public static final String WATT = "W"

Unit of power, radiant flux. It's one of the coherent derived units in the SI with special names and symbols. The unit is called watt.

141.6.10.106

public static final String WATT_PER_METER_KELVIN = "W/(m K)"

Unit of thermal conductivity. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called watt per meter kelvin.

141.6.10.107

public static final String WATT_PER_SQUARE_METER = "W/m²"

Unit of heat flux density, irradiance. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called watt per square meter.

141.6.10.108

public static final String WATT_PER_SQUARE_METER_STERADIAN = "W/(m² sr)"

Unit of radiance. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called watt per square meter steradian.

141.6.10.109

public static final String WATT_PER_STERADIAN = "W/sr"

Unit of radiant intensity. It's one of coherent derived units whose names and symbols include SI coherent derived units with special names and symbols. The unit is called watt per steradian.

141.6.10.110

public static final String WEBER = "Wb"

Unit of magnetic flux. It's one of the coherent derived units in the SI with special names and symbols. The unit is called weber.

141.7

References

- [1] *JavaBeans Spec*
<https://www.oracle.com/java/technologies/javase/javabeans-spec.html>

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT